

QUESTION NO: 29

- Write pseudocode for a home automation system showing READY, RUNNING, BLOCKED, and SUSPENDED states for tasks.

PROBLEM UNDERSTANDING:

A home automation system consists of multiple tasks such as SensorTask, ControlTask and MonitoringTask. These tasks read sensors, control appliances like lights, fans, doors and monitors system state. Tasks move through READY, RUNNING, BLOCKED and SUSPENDED states depending on execution and events.

RTOS MECHANISM USED:

Task State Management and Scheduler.

STRUCTURED PSEUDOCODE:

(A) GOAL DEFINITIONS

- DEFINE TASK SensorTask
- DEFINE TASK ControlTask
- DEFINE TASK MonitoringTask
- DEFINE QUEUE SensorQueue
- DEFINE CONSTANT QUEUE_SIZE



ASSUMPTIONS:

- The system uses an RTOS that supports task states and priority-based scheduling.
- Sensors provide valid data within a bounded time.
- Actuators (lights, fan, door lock) respond to control commands.
- Tasks communicate using queues/semaphores.
- System operates in a single microcontroller environment.
- Interrupts are used for sensor events and timer ticks.

KEY POINTS (PRIORITY ORDER):

- Task State Model: Tasks can be in READY, RUNNING, BLOCKED or SUSPENDED.
- Scheduling: Highest priority READY task is chosen to RUN.
- Blocking: Tasks wait for events/data and move to BLOCKED state.
- Synchronization: Queues/Semaphores handle communication between tasks.
- Suspension: Maintenance mode can suspend non-critical tasks.
- Real-time Operation: Critical tasks (Sensor, Control) have higher priority.

PSEUDOCODE (Contd...)

```
BEGIN
// SYSTEM INITIALIZATION
CREATE QUEUE SensorQueue WITH SIZE QUEUE_SIZE
CREATE TASK SensorTask WITH PRIORITY 3
CREATE TASK ControlTask WITH PRIORITY 2
CREATE TASK MonitoringTask WITH PRIORITY 1
SET SensorTask STATE = READY
SET ControlTask STATE = READY
SET MonitoringTask STATE = READY
START SCHEDULER
END
```

TASK SensorTask

```
BEGIN
DECLARE HomeData data
WHILE TRUE
SET SensorTask STATE = RUNNING
READ TEMPERATURE INTO data.temperature
READ LIGHT LEVEL INTO data.light_level
READ MOTION SENSOR INTO data.motion
GET CURRENT TIME INTO data.timestamp
SET SensorTask STATE = BLOCKED
WAIT UNTIL QUEUE IS NOT FULL (SensorQueue)
SET SensorTask STATE = READY
SEND data TO QUEUE SensorQueue
SET SensorTask STATE = BLOCKED
WAIT FOR 100 ms // Sampling period
SET SensorTask STATE = READY
ENDWHILE
END
```

PSEUDOCODE (Contd...)

```
TASK ControlTask
BEGIN
DECLARE HomeData received_data
WHILE TRUE
WAIT UNTIL QUEUE IS NOT EMPTY (SensorQueue)
SET ControlTask STATE = RUNNING
SET MonitoringTask STATE = RUNNING
RECEIVE received_data FROM SensorQueue
// Turn ON light level < LIGHT_THRESHOLD THEN
TURN ON LIGHT
ELSE
TURN OFF LIGHT
END IF
// Turn off fan temperature > TEMP_THRESHOLD THEN
TURN ON FAN
ELSE
TURN OFF FAN
END IF
// Turn security mode = 1 THEN
TURN ON SECURITY LIGHT
ENDWHILE
END
```

TASK MonitoringTask

```
BEGIN
WHILE TRUE
SET MonitoringTask STATE = RUNNING
DISPLAY SYSTEM STATUS
DISPLAY Temperature, Light Level, Motion Status,
Appliance Status, Time, etc.
SET MonitoringTask STATE = BLOCKED
WAIT FOR 1000 ms // Monitoring interval
SET MonitoringTask STATE = READY
ENDWHILE
END
```

PSEUDOCODE (Contd...)

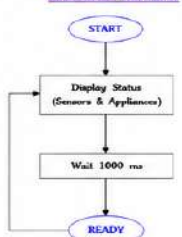
```
TASK MaintenanceTask
BEGIN
WHILE TRUE
IF maintenance_mode = ON THEN
SET ControlTask STATE = SUSPENDED
SET MonitoringTask STATE = SUSPENDED
TURN ON MAINTENANCE LIGHT
ELSE
RESUME ControlTask
RESUME MonitoringTask
TURN OFF MAINTENANCE LIGHT
END IF
WAIT FOR 2000 ms
ENDWHILE
END
```

TASK SUSPENSION

```
IF MAINTENANCE_MODE = ON THEN
SUSPEND ControlTask
SET ControlTask STATE = SUSPENDED
END IF
IF MAINTENANCE_MODE = OFF THEN
RESUME ControlTask
SET ControlTask STATE = READY
END IF
END
```

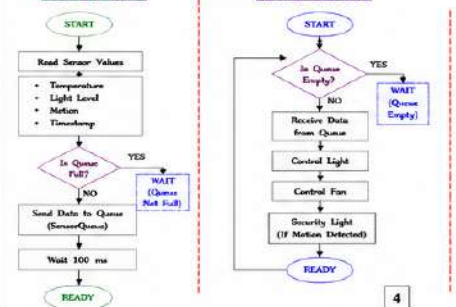
MONITORING TASK FLOW

MONITORING TASK FLOW



SENSOR TASK FLOW

CONTROL TASK FLOW



TASK STATE TRANSITION SUMMARY

Current State	Event	Next State
READY	Scheduler selects task	RUNNING
RUNNING	Task needs data / event / delay	BLOCKED
BLOCKED	Event / data available	READY
RUNNING	Preempted by higher priority task	READY
READY / RUNNING	Maintenance mode enabled	SUSPENDED
SUSPENDED	Task is resumed	READY

QUEUE / TASK OPERATIONS SUMMARY

Operation	Performed By	Purpose
SEND(data, SensorQueue)	SensorTask	Places sensor data in the queue
RECEIVE(data, SensorQueue)	ControlTask	Retrieves sensor data
WAIT(Queue Full)	SensorTask	Blocks if queue is full
WAIT(Queue Empty)	ControlTask	Blocks if queue is empty
WAIT(100 ms)	SensorTask	Sampling period
WAIT(1000 ms)	MonitoringTask	Monitoring interval
SUSPEND(ControlTask)	System/Admin	Temporarily stop task
RESUME(ControlTask)	System/Admin	Restart suspended task

STATE TRANSITION DIAGRAM



REAL TIME EXAMPLE (Home Automation System)



QUESTION NO: 29

- Write pseudocode for a home automation system showing READY, RUNNING, BLOCKED, and SUSPENDED states for tasks.

PROBLEM UNDERSTANDING:

A home automation system consists of multiple tasks such as SensorTask, ControlTask and MonitoringTask. These tasks read sensors, control appliances like lights, fans, doors and monitors system status. Tasks move through READY, RUNNING, BLOCKED and SUSPENDED states depending on execution and events.

RTOS MECHANISM USED:

Task State Management and Scheduler.

STRUCTURED PSEUDOCODE:

(A) GOAL DEFINITIONS

- DEFINE TASK SensorTask
- DEFINE TASK ControlTask
- DEFINE TASK MonitoringTask
- DEFINE QUEUE SensorQueue
- DEFINE CONSTANT QUEUE_SIZE



ASSUMPTIONS:

- The system uses an RTOS that supports task states and priority-based scheduling.
- Sensors provide valid data within a bounded time.
- Actuators (lights, fan, door lock) respond to control commands.
- Tasks communicate using queues/circular buffers.
- System operates in a single microcontroller environment.
- Interrupts are used for sensor events and timer ticks.

PSEUDOCODE (Contd...)

```
BEGIN
// SYSTEM INITIALIZATION
CREATE QUEUE SensorQueue WITH SIZE QUEUE_SIZE
CREATE TASK SensorTask WITH PRIORITY 3
CREATE TASK ControlTask WITH PRIORITY 2
CREATE TASK MonitoringTask WITH PRIORITY 1
SET SensorTask STATE = READY
SET ControlTask STATE = READY
SET MonitoringTask STATE = READY
START SCHEDULER
END
```

TASK: SensorTask

```
BEGIN
DECLARE HomeData data
WHILE TRUE
SET SensorTask STATE = RUNNING
READ TEMPERATURE INTO data.temperature
READ LIGHT LEVEL INTO data.light_level
READ MOTION SENSOR INTO data.motion
GET CURRENT TIME INTO data.timestamp
SET SensorTask STATE = BLOCKED
WAIT UNTIL QUEUE IS NOT FULL (SensorQueue)
SET SensorTask STATE = READY
SEND data TO QUEUE SensorQueue
SET SensorTask STATE = BLOCKED
WAIT FOR 100 ms // Sampling period
SET SensorTask STATE = READY
ENDWHILE
END
```

1

PSEUDOCODE (Contd...)

TASK: ControlTask

```
BEGIN
DECLARE HomeData received_data
WHILE TRUE
WAIT UNTIL QUEUE IS NOT EMPTY (SensorQueue)
SET ControlTask STATE = RUNNING
SET ControlTask STATE = RUNNING
RECEIVE received_data FROM SensorQueue
IF received_data.light_level < LIGHT_THRESHOLD THEN
TURN ON LIGHT
ELSE
TURN OFF LIGHT
END IF
IF received_data.temperature > TEMP_THRESHOLD THEN
TURN ON FAN
ELSE
TURN OFF FAN
END IF
IF received_data.motion = 1 THEN
TURN ON SECURITY LIGHT
END IF
ENDWHILE
```

TASK: MonitoringTask

```
BEGIN
WHILE TRUE
SET MonitoringTask STATE = RUNNING
DISPLAY SYSTEM STATUS
DISPLAY Temperature, Light Level, Motion Status,
Appliance Status, Time, etc.
SET MonitoringTask STATE = BLOCKED
WAIT FOR 1000 ms // Monitoring interval
SET MonitoringTask STATE = READY
ENDWHILE
END
```

2

PSEUDOCODE (Contd...)

TASK: MaintenanceTask

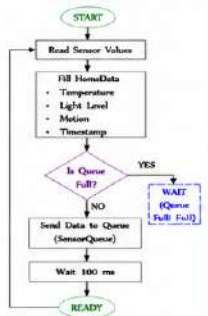
```
BEGIN
WHILE TRUE
IF maintenance_mode = ON THEN
SET ControlTask STATE = SUSPENDED
SET MonitoringTask STATE = SUSPENDED
TURN ON MAINTENANCE LIGHT
ELSE
RESUME ControlTask
RESUME MonitoringTask
TURN OFF MAINTENANCE LIGHT
END IF
END IF
WAIT FOR 2000 ms
ENDWHILE
END
```

TASK SUSPENSION

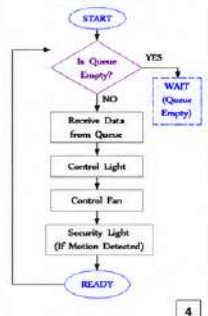
```
IF MAINTENANCE MODE = ON THEN
SUSPEND ControlTask
SUSPEND MonitoringTask
END IF
IF MAINTENANCE MODE = OFF THEN
RESUME ControlTask
RESUME MonitoringTask
SET ControlTask STATE = READY
END IF
END
```

3

SENSOR TASK FLOW

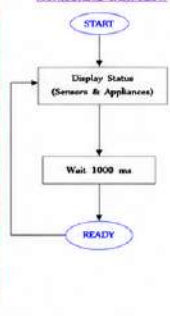


CONTROL TASK FLOW



4

MONITORING TASK FLOW



TASK STATE TRANSITION SUMMARY

Current State	Condition / Event	Next State
READY	Scheduler selects task	RUNNING
RUNNING	Task needs data / event / delay	BLOCKED
BLOCKED	Event / data available	READY
RUNNING	Preempted by higher priority task	READY
READY / RUNNING	Maintenance mode enabled	SUSPENDED
SUSPENDED	Task is resumed	READY

QUEUE / TASK OPERATIONS SUMMARY

Operation	Performed By	Purpose
SEND(data, SensorQueue)	SensorTask	Place sensor data in the queue
RECEIVE(data, SensorQueue)	ControlTask	Retrieve sensor data
WAIT(Queue Full)	SensorTask	Blocks if queue is full
WAIT(Queue Empty)	ControlTask	Blocks if queue is empty
WAIT(100 ms)	SensorTask	Sampling period
WAIT(1000 ms)	MonitoringTask	Monitoring interval
SUSPEND(ControlTask)	System/Admin	Temporarily stop task
RESUME(ControlTask)	System/Admin	Restart suspended task

5

STATE TRANSITION DIAGRAM



REAL TIME EXAMPLE (Home Automation System)



QUESTION NO: 30

- Write pseudocode for a home automation system showing task state transitions and scheduler operation.

PROBLEM UNDERSTANDING:

A home automation system performs multiple tasks such as reading sensors, controlling appliances, monitoring security and updating user interface. Tasks move through READY, RUNNING, BLOCKED and SUSPENDED states depending on events and execution.

RTOS MECHANISM USED:

Task State Management and Priority Based Preemptive Scheduler.

STRUCTURED PSEUDOCODE:

(A) GLOBAL DEFINITIONS

- DEFINE TASK SensorTask
- DEFINE TASK ControlTask
- DEFINE TASK SecurityTask
- DEFINE TASK MonitorTask
- DEFINE TASK UITask
- DEFINE QUEUE EventQueue
- DEFINE CONSTANT MAX_TASKS
- DEFINE CONSTANT TICK_MS



ASSUMPTIONS:

- The system uses an RTOS supporting task states and priority scheduling.
- Sensors provide data within a bounded time.
- Actuators respond to control commands.
- Tasks communicate using queues/semaphores.
- System operates in a single microcontroller environment.
- Interrupts are used for sensor events and timer tasks.

KEY POINTS (PRIORITY ORDER):

- Task State Model: Tasks can be in READY, RUNNING, BLOCKED or SUSPENDED.
- Scheduling: Highest priority READY task is chosen to RUN.
- Blockings: Tasks wait for events/data and move to BLOCKED state.
- Synchronization: Queues/Semaphores handle communication between tasks.
- Suspension: Maintenance mode can suspend non-critical tasks.
- Real-time Operation: Critical tasks (Sensor, Control, Security) have higher priority.

PSEUDOCODE (Contd...)

```
BEGIN
// SYSTEM INITIALIZATION
CREATE QUEUE EventQueue WITH SIZE MAX_EVENTS
CREATE TASK SensorTask WITH PRIORITY 3
CREATE TASK ControlTask WITH PRIORITY 2
CREATE TASK SecurityTask WITH PRIORITY 1
CREATE TASK MonitorTask WITH PRIORITY 4
CREATE TASK UITask WITH PRIORITY 5

SET SensorTask.state = READY
SET ControlTask.state = READY
SET MonitorTask.state = READY
SET UITask.state = READY
START SCHEDULER
END
```

TASK: SensorTask

```
BEGIN
DECLARE sensor_data

WHILE TRUE
    READ sensor values (temperature, motion, light, door)
    CREATE event from sensor_data
    SEND event TO EventQueue
    SET SensorTask.state = BLOCKED
    WAIT FOR event or TICK_MS // periodic sampling
    SET SensorTask.state = READY
ENDWHILE
END
```

PSEUDOCODE (Contd...)

TASK: ControlTask

```
BEGIN
DECLARE event

WHILE TRUE
    WAIT UNTIL EventQueue IS NOT EMPTY
    RECEIVE event FROM EventQueue
    PROCESS event
    IF event.type = LIGHT
        CONTROL light based on event.data
    ELSE IF event.type = FAN
        CONTROL fan based on event.data
    ELSE IF event.type = DOOR_LOCK
        CONTROL door lock based on event.data
    ELSE IF event.type = APPLIANCE
        CONTROL appliance based on event.data
    ENDIF
    SET ControlTask.state = READY
ENDWHILE
END
```

TASK: SecurityTask

```
BEGIN
DECLARE alert_event

WHILE TRUE
    CHECK security sensors (motion, door, smoke)
    IF threat_detected THEN
        TRIGGER alarm / notification
        SEND alert_event TO EventQueue
    ENDIF
    SET SecurityTask.state = BLOCKED
    WAIT FOR event or TICK_MS
    SET SecurityTask.state = READY
ENDWHILE
END
```

PSEUDOCODE (Contd...)

TASK: MonitorTask

```
BEGIN
DECLARE system_status

WHILE TRUE
    COLLECT system status (temp, light, power, devices)
    UPDATE logs and statistics
    DISPLAY/STORE system_status
    SET MonitorTask.state = BLOCKED
    WAIT FOR TICK_MS
    SET MonitorTask.state = READY
ENDWHILE
END
```

TASK: UITask

```
BEGIN
DECLARE user_cmd

WHILE TRUE
    WAIT FOR user command (app / keypad / voice)
    RECEIVE user_cmd
    CONVERT user_cmd TO event
    SEND event TO EventQueue
    SET UITask.state = BLOCKED
    WAIT FOR TICK_MS
    SET UITask.state = READY
ENDWHILE
END
```

SCHEDULER OPERATION (PRIORITY BASED)

```
BEGIN
LOOP FOREVER
    SELECT highest priority task from READY list
    SET selected_task.state = RUNNING
    RUN selected_task UNTIL
        (a) task BLOCKS, or
        (b) task SUSPENDED, or
        (c) higher priority task becomes READY, or
        (d) time slice (TICK_MS) expires
    IF selected task not in READY state THEN
        UPDATE its state appropriately
    ENDIF
    ENDOLOOP
END
```

TASK STATE TRANSITION SUMMARY

Current State	Condition / Event	Next State
READY	Scheduler selects task	RUNNING
RUNNING	Task blocks / waits for event	BLOCKED
RUNNING	Higher priority task ready	READY
RUNNING	Time slice expires	READY
BLOCKED	Event / data available	READY
READY / RUNNING	System in maintenance mode	SUSPENDED
SUSPENDED	Resume command received	READY

QUEUE / EVENT OPERATIONS SUMMARY

Operation	Performed By	Purpose
SEND(event)	SensorTask / UITask	Place event in EventQueue
RECEIVE(event)	ControlTask / SecurityTask	Retrieve event from queue
QUEUE FULL	System	Blocks sender task
QUEUE EMPTY	System	Blocks receiver task
PEEK(event) (optional)	MonitorTask	View next event without remove

STATE TRANSITION DIAGRAM (TASK)



REAL TIME EXAMPLE (Home Automation System)



QUESTION NO: 1

- Write pseudocode for two tasks safely incrementing a shared counter using a binary semaphore.

PROBLEM UNDERSTANDING:

Two tasks (Task1 and Task2) share a common counter. To avoid race conditions while accessing the counter, a binary semaphore (mutex) is used to ensure mutual exclusion. Only one task can access and update the counter at a time.

RTOS MECHANISM USED:

Binary Semaphore for Mutual Exclusion (Mutex).

STRUCTURED PSEUDOCODE:

(A) GLOBAL DEFINITIONS

- DEFINE SHARED INTEGER counter = 0
- DEFINE BINARY SEMAPHORE mutex
- DEFINE CONSTANT MAX_COUNT (Optional limit)
- DEFINE CONSTANT DELAY_MS (Delay between increments)

ASSUMPTIONS:

- The system uses an RTOS that supports binary semaphores.
- Both tasks have equal priority.
- Binary semaphore is initialized to 1 (available).
- Context switching can occur between any two instructions.
- Counter is a shared global variable.
- Tasks run forever, incrementing the counter periodically.

KEY POINTS (PRIORITY ORDER):

- Binary semaphore ensures mutual exclusion to the critical section.
- Semaphore is taken (wait) before accessing the shared counter.
- Semaphore is given (signal) after updating the counter.
- Prevents race condition and ensures data consistency.
- If semaphore is not available, task waits (blocks).
- After increment, task releases semaphore for the other task.



PSEUDOCODE (Contd...)

```
BEGIN
// SYSTEM INITIALIZATION
CREATE BINARY SEMAPHORE mutex WITH INITIAL COUNT = 1
CREATE TASK Task1 WITH PRIORITY 1
CREATE TASK Task2 WITH PRIORITY 1
START SCHEDULER
END
```

TASK: Task1

```
BEGIN
DECLARE temp
WHILE TRUE
    WAIT (mutex) // Take semaphore (enter critical section)
    temp = counter
    counter = temp + 1 // Update shared counter
    SIGNAL (mutex) // Give semaphore (exit critical section)
    DISPLAY "Task1 incremented counter to", counter
    DELAY (DELAY_MS)
ENDWHILE
```

PSEUDOCODE (Contd...)

```
TASK: Task2
BEGIN
DECLARE temp
WHILE TRUE
    WAIT (mutex) // Take semaphore (enter critical section)
    temp = counter
    counter = temp + 1 // Update shared counter
    SIGNAL (mutex) // Give semaphore (exit critical section)
    DISPLAY "Task2 incremented counter to", counter
    DELAY (DELAY_MS)
ENDWHILE
END
```

BINARY SEMAPHORE OPERATIONS

```
WAIT (S):
IF S > 0 THEN
    S = S - 1 // Semaphore acquired
ELSE
    Block the calling task and place it in S-wait_queue
ENDIF

SIGNAL (S):
IF S-wait_queue is not empty THEN
    Unblock highest priority task from S-wait_queue
ELSE
    S = S + 1 // Semaphore released
ENDIF
```

SCHEDULER OPERATION (HIGH LEVEL)

```
BEGIN
LOOP FOREVER
    SELECT highest priority READY task from ReadyList
    CONTEXT SWITCH to selected task
    RUN task until it:
        (a) blocks (e.g., WAIT on semaphore)
        (b) voluntarily delays
        (c) is preempted by higher priority task
    IF task is blocked THEN
        Move task to appropriate wait queue
    ELSE IF task is ready THEN
        Move task to ReadyList
    ENDIF
ENDLOOP
END
```

OUTPUT (EXAMPLE)

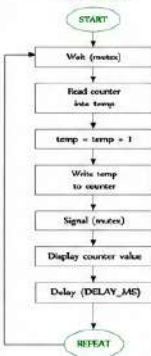
Counter value will be incremented safely by both tasks without any race condition.

Example Output (may vary):

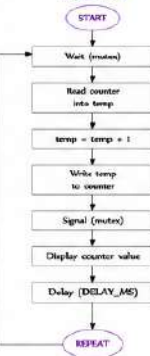
Task1 incremented counter to 1
Task2 incremented counter to 2
Task1 incremented counter to 3
Task2 incremented counter to 4

...
(Values are always correct and in sequence)

TASK1 FLOW



TASK2 FLOW



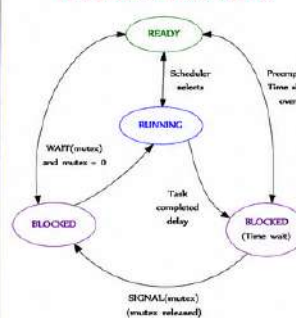
TASK STATE TRANSITION SUMMARY

Current State	Condition / Event	Next State
READY	Scheduler selects task	RUNNING
RUNNING	Calls WAIT(mutex) and mutex = 0	BLOCKED
RUNNING	Calls WAIT(mutex) and mutex > 0	RUNNING
BLOCKED	Signal(mutex) and task is unblocked	READY
RUNNING	Preempted by other task	READY
RUNNING	Calls DELAY()	BLOCKED (Time wait)
BLOCKED	Delay time expires	READY

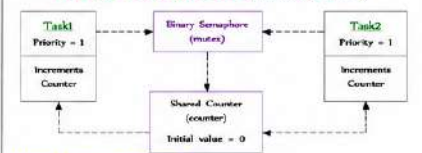
SEMAPHORE / QUEUE OPERATIONS SUMMARY

Operation	Performed By	Purpose
WAIT (mutex)	Task1 / Task2	Acquire semaphore, enter CS
SIGNAL (mutex)	Task1 / Task2	Release semaphore, exit CS
Block on WAIT	RTOS Kernel	Place task in mutex wait queue
Unblock on SIGNAL	RTOS Kernel	Move task to ReadyList

STATE TRANSITION DIAGRAM (TASK)



REAL TIME EXAMPLE (Two Tasks Incrementing Counter)



EXAMPLE EXECUTION SEQUENCE

Step	Event	Task Executing	Semaphore (mutex)	Counter Value
1	Task1 WAIT(mutex)	Task1	1 -> 0	0
2	Task1 increments counter	Task1	0	1
3	Task1 SIGNAL(mutex)	Task1	0 -> 1	1
4	Task2 WAIT(mutex)	Task2	1 -> 0	1
5	Task2 increments counter	Task2	0	2
6	Task2 SIGNAL(mutex)	Task2	0 -> 1	2
7	.. (continues)	..	..	..

QUESTION NO: 2

- Write pseudocode for a producer-consumer system using counting semaphores and a bounded buffer.

PROBLEM UNDERSTANDING:

In a producer-consumer system, producers generate items and place them into a bounded buffer. Consumers remove items from the buffer and consume them. Counting semaphores are used to ensure proper synchronization and avoid buffer overflow and underflow.

RTOS MECHANISM USED:

Counting Semaphores.

STRUCTURED PSEUDOCODE:

(A) GLOBAL DEFINITIONS

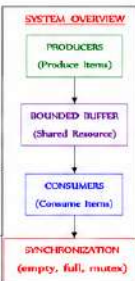
- DEFINE BUFFER_SIZE N
- DEFINE INTEGER BUFFER[BUFFER_SIZE]
- DEFINE INTEGER in = 0 // Next insertion index
- DEFINE INTEGER out = 0 // Next removal index
- DEFINE COUNTING SEMAPHORE empty // Counts empty slots
- DEFINE COUNTING SEMAPHORE full // Counts filled slots
- DEFINE BINARY SEMAPHORE mutex // Protects buffer access

ASSUMPTIONS:

- Buffer size (N) is finite and known.
- empty, full and mutex semaphores are initialized properly.
- Producers produce items at any time.
- Consumers consume items at any time.
- No busy waiting; tasks block on semaphores when needed.
- in and out are updated in a circular manner.

KEY POINTS (PRIORITY ORDER):

- empty counts available empty slots in buffer.
- full counts available filled slots in buffer.
- mutex provides mutual exclusion while accessing buffer.
- Producer waits for an empty slot before inserting.
- Consumer waits for a filled slot before removing.
- After producing/consuming, appropriate semaphore is signaled.



PSEUDOCODE (Contd...)

```

BEGIN
  // SYSTEM INITIALIZATION
  SET N = BUFFER_SIZE
  INITIALIZE in = 0
  INITIALIZE out = 0
  INITIALIZE SEMAPHORE empty = N // N empty slots initially
  INITIALIZE SEMAPHORE full = 0 // 0 filled slots initially
  INITIALIZE BINARY SEMAPHORE mutex = 1 // 1 = unlocked
  CREATE TASK ProducerTask
  CREATE TASK ConsumerTask
  START SCHEDULER
END
  
```

TASK: ProducerTask

```

BEGIN
  DECLARE item
  WHILE TRUE
    item = produce_item() // Produce an item
    WAIT(empty) // Wait for empty slot
    WAIT(mutex) // Enter critical section
    BUFFER[in] = item // Insert item into buffer
    in = (in + 1) MOD N // Update insertion index
    SIGNAL(mutex) // Exit critical section
    SIGNAL(full) // Signal that one slot is filled
  ENDWHILE
END
  
```

PSEUDOCODE (Contd...)

TASK: ConsumerTask

```

BEGIN
  DECLARE item
  WHILE TRUE
    WAIT(full) // Wait for filled slot
    WAIT(mutex) // Enter critical section
    item = BUFFER[out] // Remove item from buffer
    out = (out + 1) MOD N // Update removal index
    SIGNAL(mutex) // Exit critical section
    SIGNAL(empty) // Signal that one slot is empty
    consume_item(item) // Consume the item
  ENDWHILE
END
  
```

COUNTING SEMAPHORE OPERATIONS

```

WAIT(S):
  IF S > 0 THEN
    S = S - 1 // Resource available
  ELSE
    Block the calling task and place it in S.wait_queue
  ENDIF
SIGNAL(S):
  IF S.wait_queue is not empty THEN
    Unblock highest priority task from S.wait_queue
  ELSE
    S = S + 1 // Release resource
  ENDIF
  
```

SCHEDULER OPERATION

```

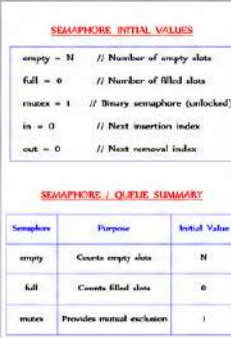
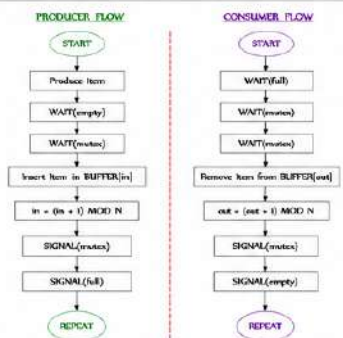
--
BEGIN
  LOOP FOREVER
    SELECT highest priority READY task from ReadyList
    CONTEXT SWITCH to selected task
    RUN task until it:
      (a) blocks on semaphore (WAIT)
      (b) yields/voluntarily delays
      (c) is preempted by higher priority task
      (d) completes
    IF task is blocked THEN
      Move task to appropriate semaphore wait queue
    ELSE IF task is READY THEN
      Move task to ReadyList
    ENDIF
  ENDLOOP
END
  
```

OUTPUT (EXAMPLE)

Assume BUFFER_SIZE = 5
 Initial: empty = 5, full = 0, mutex = 1, in = 0, out = 0
 Sequence (P = Producer, C = Consumer):

Operation	empty	full	mutex	in	out
P produces 1	4	1	0	1	0
P produces 2	3	2	0	2	0
C consumes	4	1	1	2	1
P produces 3	3	2	0	3	1
C consumes	4	1	1	3	2
C consumes	5	0	1	3	3

System continues safely without overflow/underflow.



SEMAPHORE INITIAL VALUES

```

empty = N // Number of empty slots
full = 0 // Number of filled slots
mutex = 1 // Binary semaphore (unlocked)
in = 0 // Next insertion index
out = 0 // Next removal index
  
```

SEMAPHORE / QUEUE SUMMARY

Semaphore	Purpose	Initial Value
empty	Counts empty slots	N
full	Counts filled slots	0
mutex	Provides mutual exclusion	1

TASK STATE TRANSITION SUMMARY

Current State	Condition / Event	Next State
READY	Scheduler selects task	RUNNING
RUNNING	Calls WAIT(S) and S > 0	RUNNING
RUNNING	Calls WAIT(S) and S = 0	BLOCKED
RUNNING	Calls SIGNAL(S)	RUNNING
BLOCKED	Resource released (SIGNAL)	READY
RUNNING	Task yields / Delay	READY
RUNNING	Task completes	TERMINATED

QUEUE / EVENT OPERATIONS SUMMARY

Operation	Performed By	Purpose
WAIT(empty)	Producer	Wait for empty slot
WAIT(full)	Consumer	Wait for filled slot
WAIT(mutex)	Producer/Consumer	Enter critical section
SIGNAL(mutex)	Producer/Consumer	Exit critical section
SIGNAL(full)	Producer	Indicate new item available
SIGNAL(empty)	Consumer	Indicate empty slot available

STATE TRANSITION DIAGRAM (TASK)



REAL TIME EXAMPLE (Producer-Consumer)



EXAMPLE EXECUTION SEQUENCE

Step	Action	empty	full	mutex	in	out	Buffer Status (P-S)
1	Initial State	5	0	1	0	0	[- - - -]
2	P produces A	4	1	0	1	0	[A - - -]
3	P produces B	3	2	0	2	0	[A B - -]
4	C consumes	4	1	1	2	1	[A B - -]
5	P produces C	3	2	0	3	1	[A B C -]
6	C consumes	4	1	1	3	2	[A B C -]
7	C consumes	5	0	1	3	3	[A B C -]
8	... Continue Safely ...	...	...	...	...	...	...

QUESTION NO: 3

Write pseudocode for multiple tasks accessing a shared register using a mutex with timeout.

PROBLEM UNDERSTANDING:
Multiple tasks need to read and write a shared hardware register. To avoid race conditions and ensure data integrity, a mutex is used to provide mutual exclusion. If a task cannot obtain the mutex within a specified timeout, it should handle the situation and continue without blocking indefinitely.

RISC MECHANISM USED:
Mutex with Timeout.

STRUCTURED_PSEUDOCODE:

(A) GLOBAL DEFINITIONS

- DEFINE SHARED REGISTER shared_reg
- DEFINE MUTEX reg_mutex
- DEFINE CONSTANT TIMEOUT_MS 100
- DEFINE TASK Task1 (Writer)
- DEFINE TASK Task2 (Reader)
- DEFINE TASK Task3 (Updater)
- DEFINE DELAY_MS 10

ASSUMPTIONS:

- Only one task can access the register at a time.
- Mutex provides mutual exclusion.
- If mutex is not available within timeout, task skips access.
- All tasks run at appropriate priorities.
- TIMEOUT_MS is finite and known.

KEY POINTS (PRIORITY ORDER):

- Mutex ensures exclusive access to the shared register.
- Timeout prevents tasks from waiting indefinitely.
- Task handles timeout case gracefully.
- After work, mutex must be released.
- System remains responsive even if one task misbehaves.

SYSTEM OVERVIEW

```
graph TD
    subgraph Multiple_Tasks [MULTIPLE TASKS  
(Task1, Task2, Task3)]
        direction TB
        T1[Task1]
        T2[Task2]
        T3[Task3]
    end
    subgraph Shared_Register [SHARED REGISTER  
(shared_reg)]
        direction TB
        SR[shared_reg]
    end
    subgraph Mutex [MUTEX  
(reg_mutex)]
        direction TB
        M[mutex]
    end
    T1 --> M
    T2 --> M
    T3 --> M
    M --> SR
    SR --> M
```

PROTECT ACCESS WITH MUTEX + TIMEOUT

PSEUDOCODE (Contd...)

```
BEGIN
// SYSTEM INITIALIZATION
INITIALIZE shared_reg = 0 // Initial value of register
CREATE MUTEX reg_mutex // Create mutex
CREATE TASK Task1 WITH PRIORITY 3 // Writer Task
CREATE TASK Task2 WITH PRIORITY 2 // Reader Task
CREATE TASK Task3 WITH PRIORITY 1 // Updater Task
START SCHEDULER
END
```

TASK: Task1 (Writer)

```
BEGIN
WHILE TRUE
    WAIT_MUTEX(reg_mutex, TIMEOUT_MS) // Try to get mutex
    IF mutex_acquired THEN
        shared_reg = compute_new_value() // Write to register
        DISPLAY "Task1 wrote to register"
        SIGNAL_MUTEX(reg_mutex) // Release mutex
    ELSE
        DISPLAY "Task1 timeout! Could not access register"
    ENDIF
ENDWHILE
END
```

PSEUDOCODE (Contd...)

TASK: Task2 (Reader)

```
BEGIN
WHILE TRUE
    WAIT_MUTEX(reg_mutex, TIMEOUT_MS) // Try to get mutex
    IF mutex_acquired THEN
        data = shared_reg // Read register
        DISPLAY "Task2 read register: ", data
        SIGNAL_MUTEX(reg_mutex) // Release mutex
    ELSE
        DISPLAY "Task2 timeout! Could not read register"
    ENDIF
    DELAY(DELAY_MS)
ENDWHILE
END

TASK: Task3 (Updater)



```
BEGIN
WHILE TRUE
 WAIT_MUTEX(reg_mutex, TIMEOUT_MS) // Try to get mutex
 IF mutex_acquired THEN
 shared_reg = update_register(data_source()) // Update
 DISPLAY "Task3 updated register"
 SIGNAL_MUTEX(reg_mutex) // Release mutex
 ELSE
 DISPLAY "Task3 timeout! Update skipped"
 ENDIF
 DELAY(DELAY_MS)
ENDWHILE
END
```


```

MUTEX OPERATIONS

```
WAIT_MUTEX(mutex, timeout_ms)
BEGIN
    IF mutex is available THEN
        Lock mutex
        RETURN TRUE
    ELSE
        Wait until mutex becomes available or timeout expires
        IF mutex obtained before timeout THEN
            Lock mutex
            RETURN TRUE
        ELSE
            RETURN FALSE // Timeout occurred
        ENDIF
    END
END
SIGNAL_MUTEX(mutex)
BEGIN
    Unlock mutex
END
```

OUTPUT (EXAMPLE)

Assume: TIMEOUT_MS = 100 ms

Possible Output:

- Task1 wrote to register
- Task2 read register: 85
- Task3 updated register
- Task2 timeout! Could not read register
- Task3 wrote to register
- Task3 timeout! Update skipped
- ...

(Output varies based on timing and scheduling)

TASK1 (WRITER) FLOW

TASK2 (READER) FLOW

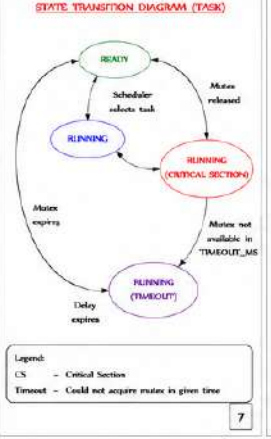
TASK3 (UPDATER) FLOW

TASK STATE TRANSITION SUMMARY

Current State	Condition / Event	Next State	Action
READY	Scheduler selects task	RUNNING	Start executing
RUNNING	Mutex acquired	RUNNING (CS)	Access register
RUNNING (CS)	Work done	RUNNING	Release mutex
RUNNING	Mutex not available within timeout	RUNNING	Timeout handling
RUNNING	Delay expires	READY	Ready for next run

MUTEX / QUEUE OPERATIONS SUMMARY

Operation	Performed By	Purpose
WAIT_MUTEX(mutex, timeout)	Task	Attempt to lock mutex with timeout
Lock mutex	Kernel	Provide mutual exclusion
SIGNAL_MUTEX(mutex)	Task	Release mutex
Timeout	Kernel	Return FALSE if timeout occurs
ReadyList/DelayList	Kernel	Manage task states



REAL TIME EXAMPLE (Multiple Tasks Accessing a Hardware Register)

Example: In an IoT device, multiple tasks need to access a configuration register of a sensor over I2C. The register must be accessed safely to prevent data corruption.

How it works:

- All tasks request access to the register using reg_mutex.
- Only one task can access the register at a time.
- If mutex is not available within TIMEOUT_MS, task skips the operation.
- Ensures data integrity and system responsiveness.